

PAULT DOCUMENTATION

2026 02 15 Creation Window Variables Richmedia

Source: docs/plans/2026-02-15-creation-window-variables-richmedia.md

Theme: Clean Technical

Generated: February 26, 2026

New Prompt Window, Auto-Expanding Variables, Rich Media — Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Add a dedicated creation window, auto-expanding variable text fields, and rich media attachments (inline images, attachments strip, rich clipboard) to Paul.

Architecture: Three features layered in dependency order. Feature 2 (auto-expanding fields) is standalone. Feature 1 (creation window) uses SwiftUI Window scene with `@Environment(\.openWindow)`. Feature 3 (rich media) introduces an `Attachment SwiftData` model, replaces `TextEditor` with an `NSTextView`-backed rich text editor, adds an attachments strip, and upgrades clipboard to RTFD.

Tech Stack: SwiftUI, SwiftData, AppKit (`NSTextView`, `NSTextAttachment`, `NSPasteboard`), `NSViewRepresentable`, security-scoped bookmarks.

Feature 2: Auto-Expanding Variable Fields

Task 1: Create `ExpandingTextEditor` component

Files:

- Create: `Pault/ExpandingTextEditor.swift`
- Test: Manual — resize behavior is visual

Step 1: Create the `NSViewRepresentable` wrapper

Create `Pault/ExpandingTextEditor.swift`:

```
import SwiftUI
import AppKit

struct ExpandingTextEditor: NSViewRepresentable {
    @Binding var text: String
    var placeholder: String = ""
    var font: NSFont = .systemFont(ofSize: NSFont.systemFontSize)

    func makeCoordinator() -> Coordinator {
        Coordinator(self)
    }

    func makeNSView(context: Context) -> NSScrollView {
        let scrollView = NSScrollView()
        let textView = NSTextView()

        textView.delegate = context.coordinator
        textView.font = font
        textView.string = text
        textView.isRichText = false
        textView.allowsUndo = true
        textView.isEditable = true
        textView.isSelectable = true
    }
}
```

```
// Auto-expanding behavior
textView.isVerticallyResizable = true
textView.isHorizontallyResizable = false
textView.autoresizingMask = [.width]
textView.textContainer?.containerSize = NSSize(width: 0, height:
CGFloat.greatestFiniteMagnitude)
textView.textContainer?.widthTracksTextView = true

// Styling
textView.textContainerInset = NSSize(width: 4, height: 6)
textView.backgroundColor = .textBackgroundColor
textView.drawsBackground = true

// Placeholder
if text.isEmpty {
    textView.placeholderAttributedString = NSAttributedString(
        string: placeholder,
        attributes: [
            .foregroundColor: NSColor.placeholderTextColor,
            .font: font
        ]
    )
}

scrollView.documentView = textView
scrollView.hasVerticalScroller = false
scrollView.hasHorizontalScroller = false
scrollView.drawsBackground = false
scrollView.borderType = .noBorder

// Track height changes
context.coordinator.textView = textView
context.coordinator.scrollView = scrollView

NotificationCenter.default.addObserver(
    context.coordinator,
    selector: #selector(Coordinator.textDidChange(_)),
    name: NSText.didChangeNotification,
    object: textView
)

return scrollView
}

func updateNSView(_ scrollView: NSScrollView, context: Context) {
    guard let textView = scrollView.documentView as? NSTextView else { return }
    if textView.string != text {
        textView.string = text
        context.coordinator.updateHeight()
    }
}

// Update placeholder
if text.isEmpty {
    textView.placeholderAttributedString = NSAttributedString(
        string: placeholder,
        attributes: [
            .foregroundColor: NSColor.placeholderTextColor,
```

```

.font: font
]
)
}
}

class Coordinator: NSObject, NSTextViewDelegate {
var parent: ExpandingTextEditor
weak var textView: NSTextView?
weak var scrollView: NSScrollView?
@Published var height: CGFloat = 30

init(_ parent: ExpandingTextEditor) {
self.parent = parent
}

@objc func textDidChange(_ notification: Notification) {
guard let textView = textView else { return }
parent.text = textView.string
updateHeight()
}

func updateHeight() {
guard let textView = textView, let scrollView = scrollView else { return }
guard let layoutManager = textView.layoutManager,
let textContainer = textView.textContainer else { return }

layoutManager.ensureLayout(for: textContainer)
let usedRect = layoutManager.usedRect(for: textContainer)
let inset = textView.textContainerInset
let newHeight = max(usedRect.height + inset.height * 2, 30)

if abs(scrollView.frame.height - newHeight) > 1 {
scrollView.invalidateIntrinsicContentSize()
}
}
}
}

```

Step 2: Build to verify compilation

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Commit

```

git add Pault/ExpandingTextEditor.swift
git commit -m "feat: add ExpandingTextEditor NSViewRepresentable component"

```

Task 2: Replace TextField with ExpandingTextEditor in TemplateVariablesView

Files:

- Modify: `Pault/TemplateVariablesView.swift:60-78`

Step 1: Replace the TextField binding in the Grid

In `Pault/TemplateVariablesView.swift`, replace the Grid block (lines 60-78) with:

```

Grid(alignment: .leading, horizontalSpacing: 12, verticalSpacing: 8) {
  ForEach(sortedVariables) { variable in
    GridRow {
      Text(variable.name)
        .font(.body.monospaced())
        .foregroundColor(.secondary)
        .frame(minWidth: 80, alignment: .trailing)
        .padding(.top, 6)

      ExpandingTextEditor(
        text: Binding(
          get: { variable.defaultValue },
          set: { newValue in
            variable.defaultValue = newValue
            debouncedSave()
          }
        ),
        placeholder: "Enter \((variable.name))..."
      )
        .frame(minHeight: 30)
        .overlay(
          RoundedRectangle(cornerRadius: 6)
            .stroke(Color.secondary.opacity(0.3), lineWidth: 1)
        )
    }
  }
}

```

Step 2: Build and verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Manual test

- Open Pault
- Create or select a prompt with `{{variable}}` in content
- Type in variable field — verify it starts single-line
- Enter multiple lines or long text — verify the field grows
- Verify text persists after switching prompts

Step 4: Commit

```

git add Pault/TemplateVariablesView.swift
git commit -m "feat: use auto-expanding text fields for template variables"

```

Feature 1: New Prompt Creation Window

Task 3: Add Window scene to PaultApp

Files:

- Modify: `Pault/PaultApp.swift:47-66`

Step 1: Add a new Window scene

In `Pault/PaultApp.swift`, add a second scene after the existing `WindowGroup`:

```
var body: some Scene {
    WindowGroup {
        ContentView()
    }
    .windowResizability(.contentSize)
    .defaultSize(width: 900, height: 600)
    .modelContainer(sharedModelContainer)
    .commands {
        CommandGroup(replacing: .newItem) {
            Button("New Prompt") {
                NotificationCenter.default.post(name: .createNewPrompt, object: nil)
            }
            .keyboardShortcut("n", modifiers: .command)
        }
    }

    Window("New Prompt", id: "new-prompt") {
        NewPromptView()
    }
    .windowResizability(.contentMinSize)
    .defaultSize(width: 600, height: 500)
    .modelContainer(sharedModelContainer)

    Settings {
        PreferencesView()
    }
}
```

Step 2: Update the Cmd+N command to open the new window instead of inline creation

In `Pault/PaultApp.swift`, change the `CommandGroup` to use `openWindow`:

We need to use `@Environment(\.openWindow)` but that's only available in views, not in App. Instead, keep the notification approach but change the handler in `ContentView` to open the window.

Actually, the cleanest approach: add the `openWindow` environment to the `ContentView` and forward the notification there.

In `Pault/ContentView.swift`, replace the `onReceive` handler and add `openWindow`:

```
// Add at the top of ContentView, after the existing @State properties:
@Environment(\.openWindow) private var openWindow

// Replace the existing .onReceive block:
.onReceive(NotificationCenter.default.publisher(for: .createNewPrompt)) { _ in
    openWindow(id: "new-prompt")
}
```

Remove the `createNewPrompt()` call from the toolbar "plus" button action as well, replacing it with `openWindow(id: "new-prompt")`.

Also remove the old `createNewPrompt()` private method from `ContentView` — it's no longer needed (the creation window handles insert now).

Step 3: Build to verify

```
Run: xcodebuild build -project Pault.xcodeproj -scheme Pault -destination  
'platform=macOS' 2>&1 | tail -5
```

Expected: BUILD SUCCEEDED (with warning about missing NewPromptView — we'll create it next)

Step 4: Commit

```
git add Pault/PaultApp.swift Pault/ContentView.swift  
git commit -m "feat: add Window scene for prompt creation, wire ■N"
```

Task 4: Create NewPromptView

Files:

- Create: Pault/NewPromptView.swift

Step 1: Write the creation window view

Create Pault/NewPromptView.swift:

```
import SwiftUI  
import SwiftData  
  
struct NewPromptView: View {  
    @Environment(\.modelContext) private var modelContext  
    @Environment(\.dismiss) private var dismiss  
    @Query(sort: [SortDescriptor(\Tag.name, order: .forward)]) private var allTags: [Tag]  
  
    @State private var title: String = ""  
    @State private var content: String = ""  
    @State private var selectedTags: [Tag] = []  
    @State private var showingTagPicker: Bool = false  
  
    private var service: PromptService { PromptService(modelContext: modelContext) }  
  
    private var parsedVariableNames: [String] {  
        TemplateEngine.extractVariableNames(from: content)  
    }  
  
    var body: some View {  
        VStack(alignment: .leading, spacing: 0) {  
            // Title  
            TextField("Prompt title", text: $title)  
                .font(.title2)  
                .fontWeight(.semibold)  
                .textFieldStyle(.plain)  
                .padding(.horizontal, 20)  
                .padding(.top, 20)  
                .padding(.bottom, 12)  
  
            // Content editor  
            TextEditor(text: $content)  
                .font(.body)  
                .scrollContentBackground(.hidden)  
                .padding(.horizontal, 16)  
                .frame(minHeight: 200)  
  
            // Tags section  
            VStack(alignment: .leading, spacing: 8) {  
                Divider()  

```

```

.padding(.horizontal, 16)

HStack(spacing: 8) {
  Label("Tags", systemImage: "tag")
    .font(.headline)
    .foregroundColor(.secondary)

  FlowLayout(spacing: 6) {
    ForEach(selectedTags) { tag in
      TagPillView(name: tag.name, color: tag.color, onRemove: {
        selectedTags.removeAll { $0.id == tag.id }
      })
    }
  }

  Button(action: { showingTagPicker.toggle() }) {
    Image(systemName: "plus")
      .font(.caption)
      .padding(6)
      .background(Color.secondary.opacity(0.2))
      .clipShape(Circle())
  }
  .buttonStyle(.plain)
  .popover(isPresented: $showingTagPicker) {
    TagPickerPopover(
      allTags: allTags,
      selectedTags: selectedTags,
      onSelect: { tag in
        if !selectedTags.contains(where: { $0.id == tag.id }) {
          selectedTags.append(tag)
        }
        showingTagPicker = false
      },
      onCreate: { name, color in
        let tag = service.createTag(name: name, color: color)
        selectedTags.append(tag)
        showingTagPicker = false
      }
    )
    .frame(width: 200, height: 300)
  }
}
.padding(.horizontal, 20)

// Template variables indicator
if !parsedVariableNames.isEmpty {
  HStack(spacing: 6) {
    Image(systemName: "curlybraces")
      .foregroundColor(.secondary)
    Text("\(parsedVariableNames.count) variable\(parsedVariableNames.count == 1 ? "" : "s")
      detected: ")
      .foregroundColor(.secondary)
    Text(parsedVariableNames.joined(separator: ", "))
      .font(.body.monospaced())
      .foregroundColor(.blue)
  }
  .font(.caption)

```



```

.padding(.horizontal, 20)
.padding(.top, 8)
}

Spacer()

// Action buttons
Divider()

HStack {
  Spacer()
  Button("Cancel") {
    dismiss()
  }
  .keyboardShortcut(.cancelAction)

  Button("Create Prompt") {
    createPrompt()
  }
  .keyboardShortcut(.defaultAction)
  .disabled(title.isEmpty && content.isEmpty)
}
.padding(16)
}
.frame(minWidth: 500, idealWidth: 600, minHeight: 400, idealHeight: 500)
}

private func createPrompt() {
  let prompt = service.createPrompt(title: title, content: content)
  for tag in selectedTags {
    service.addTag(tag, to: prompt)
  }
  TemplateEngine.syncVariables(for: prompt, in: modelContext)
  NotificationCenter.default.post(
    name: .promptCreated,
    object: nil,
    userInfo: ["promptID": prompt.id]
  )
  dismiss()
}
}

```

Step 2: Add the .promptCreated notification name

In `Pault/PaultApp.swift`, add alongside the existing notification:

```

extension Notification.Name {
  static let createNewPrompt = Notification.Name("com.pault.createNewPrompt")
  static let promptCreated = Notification.Name("com.pault.promptCreated")
}

```

Step 3: Handle the notification in ContentView to auto-select the new prompt

In `Pault/ContentView.swift`, add a second `.onReceive`:

```

.onReceive(NotificationCenter.default.publisher(for: .promptCreated)) { notification in
  if let promptID = notification.userInfo?["promptID"] as? UUID {
    selectedPrompt = prompts.first { $0.id == promptID }
  }
}

```

```
}
```

Step 4: Build and verify

```
Run: xcodebuild build -project Pault.xcodeproj -scheme Pault -destination
'platform=macOS' 2>&1 | tail -5
```

Expected: BUILD SUCCEEDED

Step 5: Commit

```
git add Pault/NewPromptView.swift Pault/PaultApp.swift Pault/ContentView.swift
git commit -m "feat: add NewPromptView creation window with tags and variable detection"
```

Task 5: Extract TagPickerPopover for reuse

The TagPickerView in InspectorView.swift is private. We need a shared version for both InspectorView and NewPromptView.

Files:

- Create: Pault/TagPickerPopover.swift
- Modify: Pault/InspectorView.swift:137-211 — remove private TagPickerView, use TagPickerPopover

Step 1: Create shared TagPickerPopover

Create Pault/TagPickerPopover.swift — extract the private TagPickerView from InspectorView.swift and rename it to TagPickerPopover:

```
import SwiftUI

struct TagPickerPopover: View {
    let allTags: [Tag]
    let selectedTags: [Tag]
    let onSelect: (Tag) -> Void
    let onCreate: (String, String) -> Void

    @State private var newTagName: String = ""
    @State private var selectedColor: String = "blue"

    private let colors = TagColors.all

    private var availableTags: [Tag] {
        allTags.filter { tag in
            !selectedTags.contains(where: { $0.id == tag.id })
        }
    }

    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Text("Add Tag")
                .font(.headline)

            if !availableTags.isEmpty {
                ScrollView {
                    VStack(spacing: 4) {
                        ForEach(availableTags) { tag in
                            Button(action: { onSelect(tag) }) {

```

```

HStack {
  TagPillView(name: tag.name, color: tag.color, isSmall: true)
  Spacer()
}
}
.buttonStyle(.plain)
}
}
}
.frame(maxHeight: 120)

Divider()
}

Text("Create New")
  .font(.subheadline)
  .foregroundColor(.secondary)

TextField("Tag name", text: $newTagName)
  .textFieldStyle(.roundedBorder)

HStack(spacing: 4) {
  ForEach(colors, id: \.self) { color in
    Circle()
      .fill(TagColors.color(for: color))
      .frame(width: 20, height: 20)
      .overlay(
        Circle()
          .stroke(Color.primary, lineWidth: selectedColor == color ? 2 : 0)
      )
      .onTapGesture {
        selectedColor = color
      }
  }
}

Button("Create") {
  let trimmed = newTagName.trimmingCharacters(in: .whitespacesAndNewlines)
  guard !trimmed.isEmpty else { return }
  onCreate(trimmed, selectedColor)
  newTagName = ""
}
.disabled(newTagName.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
}
.padding()
}
}

```

Step 2: Update InspectorView to use TagPickerPopover

In Pault/InspectorView.swift:

- Delete the private struct TagPickerView (lines 137-211)
- Replace the .popover usage at line 44 — change TagPickerView(to TagPickerPopover(and rename parameter existingTags to allTags

Step 3: Build and run tests

```
Run: xcodebuild build -project Pault.xcodeproj -scheme Pault -destination
'platform=macOS' 2>&1 | tail -5
```

```
Run: xcodebuild test -project Pault.xcodeproj -scheme Pault -destination
'platform=macOS' -only-testing PaultTests 2>&1 | tail -10
```

Expected: BUILD SUCCEEDED, all tests pass

Step 4: Commit

```
git add Pault/TagPickerPopover.swift Pault/InspectorView.swift
git commit -m "refactor: extract TagPickerPopover for reuse across views"
```

Task 6: Run all tests and verify no regressions

Step 1: Run tests

```
Run: xcodebuild test -project Pault.xcodeproj -scheme Pault -destination
'platform=macOS' -only-testing PaultTests 2>&1 | tail -20
```

Expected: All 22+ tests pass

Step 2: Manual test

- Press Cmd+N — new window opens (not inline creation)
- Fill title, content, tags
- Type `{{name}}` in content — variable indicator appears
- Click "Create Prompt" — window closes, new prompt is selected in sidebar
- Press Cancel — window closes, no prompt created

Step 3: Commit tag

```
git commit --allow-empty -m "checkpoint: Feature 1 and 2 complete — creation window and
expanding fields"
```

Feature 3: Rich Media Attachments

Task 7: Create Attachment model

Files:

- Create: Pault/Attachment.swift
- Modify: Pault/Prompt.swift
- Modify: Pault/PaultApp.swift
- Modify: Pault/ContentView.swift (preview container)

Step 1: Write the failing test

Create PaultTests/AttachmentTests.swift:

```
import Testing
import SwiftData
@testable import Pault

struct AttachmentTests {

    @Test func attachmentInitDefaults() async throws {
        let attachment = Attachment(filename: "photo.jpg", mediaType: "public.jpeg", fileSize:
```

```

1024)
#expect(attachment.filename == "photo.jpg")
#expect(attachment.mediaType == "public.jpeg")
#expect(attachment.fileSize == 1024)
#expect(attachment.storageMode == "embedded")
#expect(attachment.relativePath == nil)
#expect(attachment.bookmarkData == nil)
#expect(attachment.thumbnailData == nil)
#expect(attachment.sortOrder == 0)
}

@Test func promptStartsWithNoAttachments() async throws {
    let prompt = Prompt(title: "Test", content: "Content")
    #expect(prompt.attachments.isEmpty)
}

@Test func cascadeDeleteRemovesAttachments() async throws {
    let config = ModelConfiguration(isStoredInMemoryOnly: true)
    let container = try ModelContainer(
        for: Prompt.self, Attachment.self, Tag.self, TemplateVariable.self,
        configurations: config
    )
    let context = container.mainContext

    let prompt = Prompt(title: "Test", content: "Content")
    context.insert(prompt)

    let attachment = Attachment(filename: "test.png", mediaType: "public.png", fileSize:
512)
    context.insert(attachment)
    prompt.attachments = [attachment]
    try context.save()

    context.delete(prompt)
    try context.save()

    let remaining = try context.fetch(FetchDescriptor<Attachment>())
    #expect(remaining.isEmpty)
}
}

```

Step 2: Run test to verify it fails

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' -only-testing PaultTests/AttachmentTests 2>&1 | tail -10`

Expected: FAIL — Attachment not found

Step 3: Create the Attachment model

Create `Pault/Attachment.swift`:

```

import Foundation
import SwiftData

@Model
final class Attachment {
    var id: UUID
    var filename: String
    var mediaType: String

```

```

var fileSize: Int64
var storageMode: String
var relativePath: String?
var bookmarkData: Data?
var thumbnailData: Data?
var sortOrder: Int
var createdAt: Date
@Relationship var prompt: Prompt?

init(
  id: UUID = UUID(),
  filename: String,
  mediaType: String,
  fileSize: Int64,
  storageMode: String = "embedded",
  relativePath: String? = nil,
  bookmarkData: Data? = nil,
  thumbnailData: Data? = nil,
  sortOrder: Int = 0,
  createdAt: Date = Date()
) {
  self.id = id
  self.filename = filename
  self.mediaType = mediaType
  self.fileSize = fileSize
  self.storageMode = storageMode
  self.relativePath = relativePath
  self.bookmarkData = bookmarkData
  self.thumbnailData = thumbnailData
  self.sortOrder = sortOrder
  self.createdAt = createdAt
}
}

```

Step 4: Add attachments relationship to Prompt

In Pault/Prompt.swift, add after the templateVariables relationship:

```

@Relationship(deleteRule: .cascade, inverse: \Attachment.prompt) var attachments:
[Attachment]

```

Add attachments: [Attachment] = [] to the init parameters and self.attachments = attachments in the body.

Step 5: Add Attachment to Schema

In Pault/PaultApp.swift, add Attachment.self to the Schema array.

In Pault/ContentView.swift, add Attachment.self to the #Preview modelContainer.

Step 6: Run tests

```

Run: xcodebuild test -project Pault.xcodeproj -scheme Pault -destination
'platform=macOS' -only-testing PaultTests 2>&1 | tail -20

```

Expected: All tests pass including new AttachmentTests

Step 7: Commit

```
git add Pault/Attachment.swift Pault/Prompt.swift Pault/PaultApp.swift
Pault/ContentView.swift PaultTests/AttachmentManagerTests.swift
git commit -m "feat: add Attachment model with cascade delete from Prompt"
```

Task 8: Create AttachmentManager for file storage

Files:

- Create: Pault/AttachmentManager.swift
- Test: PaultTests/AttachmentManagerTests.swift

Step 1: Write the failing test

Create PaultTests/AttachmentManagerTests.swift:

```
import Testing
import Foundation
@testable import Pault

struct AttachmentManagerTests {

    @Test func attachmentsDirectoryExists() async throws {
        let dir = AttachmentManager.attachmentsBaseDirectory
        #expect(dir.pathExtension == "")
        #expect(dir.lastPathComponent == "Attachments")
    }

    @Test func promptDirectoryUsesPromptID() async throws {
        let id = UUID()
        let dir = AttachmentManager.directory(for: id)
        #expect(dir.lastPathComponent == id.uuidString)
    }

    @Test func sizeThresholdIs10MB() async throws {
        #expect(AttachmentManager.embeddedSizeThreshold == 10 * 1024 * 1024)
    }
}
```

Step 2: Run test to verify it fails

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' -only-testing PaultTests/AttachmentManagerTests 2>&1 | tail -10`

Expected: FAIL — AttachmentManager not found

Step 3: Implement AttachmentManager

Create Pault/AttachmentManager.swift:

```
import Foundation
import AppKit
import UniformTypeIdentifiers
import os

private let attachLogger = Logger(subsystem: "com.pault.app", category:
"AttachmentManager")

enum AttachmentManager {

    static let embeddedSizeThreshold: Int64 = 10 * 1024 * 1024 // 10 MB
```

```

static var attachmentsBaseDirectory: URL {
let appSupport = FileManager.default.urls(for: .applicationSupportDirectory, in:
.userDomainMask).first!
return appSupport.appendingPathComponent("Pault/Attachments", isDirectory: true)
}

static func directory(for promptID: UUID) -> URL {
attachmentsBaseDirectory.appendingPathComponent(promptID.uuidString, isDirectory: true)
}

/// Store a file for a prompt. Returns a configured Attachment (not yet inserted into
context).
static func storeFile(at sourceURL: URL, for promptID: UUID) throws -> Attachment {
let filename = sourceURL.lastPathComponent
let fileSize = try Int64(FileManager.default.attributesOfItem(atPath:
sourceURL.path)[.size] as? UInt64 ?? 0)
let uti = UTType(filenameExtension: sourceURL.pathExtension)?.identifier ??
"public.data"

if fileSize <= embeddedSizeThreshold {
return try storeEmbedded(sourceURL: sourceURL, promptID: promptID, filename: filename,
mediaType: uti, fileSize: fileSize)
} else {
return try storeReferenced(sourceURL: sourceURL, filename: filename, mediaType: uti,
fileSize: fileSize)
}
}

private static func storeEmbedded(sourceURL: URL, promptID: UUID, filename: String,
mediaType: String, fileSize: Int64) throws -> Attachment {
let destDir = directory(for: promptID)
try FileManager.default.createDirectory(at: destDir, withIntermediateDirectories: true)

let attachmentID = UUID()
let ext = sourceURL.pathExtension
let destURL = destDir.appendingPathComponent("\(attachmentID.uuidString).\(ext)")

try FileManager.default.copyItem(at: sourceURL, to: destURL)
attachLogger.info("Embedded file: \(filename) (\(fileSize) bytes)")

let relativePath = "\(promptID.uuidString)/\(attachmentID.uuidString).\(ext)"
return Attachment(
id: attachmentID,
filename: filename,
mediaType: mediaType,
fileSize: fileSize,
storageMode: "embedded",
relativePath: relativePath
)
}

private static func storeReferenced(sourceURL: URL, filename: String, mediaType: String,
fileSize: Int64) throws -> Attachment {
let bookmarkData = try sourceURL.bookmarkData(
options: [.withSecurityScope, .securityScopeAllowOnlyReadAccess],
includingResourceValuesForKeys: nil,
relativeTo: nil

```



```

)
attachLogger.info("Referenced file: \(filename) \(fileSize) bytes")

return Attachment(
  filename: filename,
  mediaType: mediaType,
  fileSize: fileSize,
  storageMode: "referenced",
  bookmarkData: bookmarkData
)
}

/// Resolve the on-disk URL for an attachment.
static func resolveURL(for attachment: Attachment) -> URL? {
  switch attachment.storageMode {
  case "embedded":
    guard let relativePath = attachment.relativePath else { return nil }
    return attachmentsBaseDirectory.appendingPathComponent(relativePath)
  case "referenced":
    guard let bookmarkData = attachment.bookmarkData else { return nil }
    var isStale = false
    guard let url = try? URL(resolvingBookmarkData: bookmarkData, options:
      .withSecurityScope, relativeTo: nil, bookmarkDataIsStale: &isStale) else { return nil }
    if isStale {
      attachLogger.warning("Stale bookmark for: \(attachment.filename)")
    }
    _ = url.startAccessingSecurityScopedResource()
    return url
  default:
    return nil
  }
}

/// Delete embedded files for a prompt from disk.
static func deleteFiles(for promptID: UUID) {
  let dir = directory(for: promptID)
  try? FileManager.default.removeItem(at: dir)
}

/// Delete a single embedded attachment file from disk.
static func deleteFile(for attachment: Attachment) {
  guard attachment.storageMode == "embedded", let relativePath = attachment.relativePath
  else { return }
  let url = attachmentsBaseDirectory.appendingPathComponent(relativePath)
  try? FileManager.default.removeItem(at: url)
}

/// Generate a thumbnail for an image file.
static func generateThumbnail(for url: URL, maxSize: CGFloat = 120) -> Data? {
  guard let image = UIImage(contentsOf: url) else { return nil }
  let size = image.size
  let scale = min(maxSize / size.width, maxSize / size.height, 1.0)
  let newSize = NSSize(width: size.width * scale, height: size.height * scale)

  let thumbnail = UIImage(size: newSize)
  thumbnail.lockFocus()
  image.draw(in: NSRect(origin: .zero, size: newSize))
  thumbnail.unlockFocus()
}

```

```

guard let tiffData = thumbnail.tiffRepresentation,
let rep = NSBitmapImageRep(data: tiffData) else { return nil }
return rep.representation(using: .jpeg, properties: [.compressionFactor: 0.7])
}

/// Check if a UTI represents an image type.
static func isImage(_ mediaType: String) -> Bool {
guard let uti = UTType(mediaType) else { return false }
return uti.conforms(to: .image)
}
}

```

Step 4: Run tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' -only-testing PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 5: Commit

```

git add Pault/AttachmentManager.swift PaultTests/AttachmentManagerTests.swift
git commit -m "feat: add AttachmentManager for hybrid file storage"

```

Task 9: Create RichTextEditor (NSTextView wrapper)

Files:

- Create: `Pault/RichTextEditor.swift`

Step 1: Create the NSViewRepresentable

Create `Pault/RichTextEditor.swift`:

```

import SwiftUI
import AppKit

struct RichTextEditor: NSViewRepresentable {
@Binding var attributedContent: Data?
@Binding var plainContent: String
var onImageDrop: ((URL) -> Void)?

func makeCoordinator() -> Coordinator {
Coordinator(self)
}

func makeNSView(context: Context) -> NSScrollView {
let scrollView = NSScrollView()
let textView = NSTextView()

textView.delegate = context.coordinator
textView.isRichText = true
textView.allowsUndo = true
textView.isEditable = true
textView.isSelectable = true
textView.importsGraphics = true
textView.allowsImageEditing = false
textView.usesRuler = false

```

```

// Text container setup
textView.isVerticallyResizable = true
textView.isHorizontallyResizable = false
textView.autoresizingMask = [.width]
textView.textContainer?.containerSize = NSSize(width: 0, height:
CGFloat.greatestFiniteMagnitude)
textView.textContainer?.widthTracksTextView = true
textView.textContainerInset = NSSize(width: 4, height: 8)

// Font
textView.font = .systemFont(ofSize: NSFont.systemFontSize)
textView.backgroundColor = .clear

// Drag and drop for images
textView.registerForDraggedTypes([.fileURL, .png, .tiff])

scrollView.documentView = textView
scrollView.hasVerticalScroller = true
scrollView.hasHorizontalScroller = false
scrollView.drawsBackground = false
scrollView.borderType = .noBorder
scrollView.autohidesScrollers = true

context.coordinator.textView = textView

// Load initial content
if let data = attributedContent,
let attrString = try? NSAttributedString(data: data, options: [.documentType:
NSAttributedString.DocumentType.rtf], documentAttributes: nil) {
textView.textStorage?.setAttributedString(attrString)
} else if !plainContent.isEmpty {
textView.string = plainContent
}

return scrollView
}

func updateNSView(_ scrollView: NSScrollView, context: Context) {
// Only update from external changes (avoid feedback loop)
guard !context.coordinator.isEditing else { return }
guard let textView = scrollView.documentView as? NSTextView else { return }

if let data = attributedContent,
let attrString = try? NSAttributedString(data: data, options: [.documentType:
NSAttributedString.DocumentType.rtf], documentAttributes: nil) {
if textView.attributedString() != attrString {
textView.textStorage?.setAttributedString(attrString)
}
}
}

class Coordinator: NSObject, NSTextViewDelegate {
var parent: RichTextEditor
weak var textView: NSTextView?
var isEditing: Bool = false

init(_ parent: RichTextEditor) {
self.parent = parent

```

```

}

func textDidBeginEditing(_ notification: Notification) {
    isEditing = true
}

func textDidEndEditing(_ notification: Notification) {
    isEditing = false
    syncContent()
}

func textDidChange(_ notification: Notification) {
    syncContent()
}

private func syncContent() {
    guard let textView = textView else { return }

    // Sync plain text
    parent.plainContent = textView.string

    // Sync attributed content as RTFD data
    let fullRange = NSRange(location: 0, length: textView.textStorage?.length ?? 0)
    if let textStorage = textView.textStorage {
        parent.attributedContent = try? textStorage.data(
            from: fullRange,
            documentAttributes: [.documentType: NSAttributedString.DocumentType.rtf]
        )
    }
}

// Handle drag and drop of image files
func textView(_ textView: NSTextView, draggedCell cell: NSTextAttachmentCell, in rect:
    NSRange, event: NSEvent?, at charIndex: Int) {
    // Default behavior
}
}
}

```

Step 2: Build to verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Commit

```

git add Pault/RichTextEditor.swift
git commit -m "feat: add RichTextEditor NSViewRepresentable for rich text editing"

```

Task 10: Add attributedContent to Prompt model

Files:

- Modify: `Pault/Prompt.swift`

Step 1: Add the attributedContent property

In `Pault/Prompt.swift`, add after the content property:

```
var attributedContent: Data?
```

Add `attributedContent: Data? = nil` to the init parameters and `self.attributedContent = attributedContent` in the body.

Step 2: Build and run tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' -only-testing PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 3: Commit

```
git add Pault/Prompt.swift
git commit -m "feat: add attributedContent (RTFD data) to Prompt model"
```

Task 11: Replace `TextEditor` with `RichTextEditor` in `PromptDetailView`

Files:

- Modify: `Pault/PromptDetailView.swift:42-51`

Step 1: Replace `TextEditor`

In `Pault/PromptDetailView.swift`, replace the `TextEditor` block (lines 43-51) with:

```
RichTextEditor(
    attributedContent: $prompt.attributedContent,
    plainContent: $prompt.content
)
.padding(.horizontal, 16)
.padding(.bottom, 16)
.onChange(of: prompt.content) { _, _ in
    debouncedSave()
    debouncedSyncVariables()
}
.onChange(of: prompt.attributedContent) { _, _ in
    debouncedSave()
}
```

Step 2: Build and verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Manual test

- Open Pault, select a prompt
- Type text — verify it appears and saves
- Paste an image (Cmd+V) — verify it appears inline
- Template variables still work

Step 4: Commit

```
git add Pault/PromptDetailView.swift
git commit -m "feat: replace TextEditor with RichTextEditor in detail view"
```

Task 12: Create `AttachmentsStripView`

Files:

- Create: Pault/AttachmentsStripView.swift

Step 1: Create the strip view

Create Pault/AttachmentsStripView.swift:

```
import SwiftUI
import SwiftData
import UniformTypeIdentifiers

struct AttachmentsStripView: View {
    @Environment(\.modelContext) private var modelContext
    @Bindable var prompt: Prompt

    @State private var dragOver: Bool = false

    private var sortedAttachments: [Attachment] {
        prompt.attachments.sorted { $0.sortOrder < $1.sortOrder }
    }

    var body: some View {
        if !prompt.attachments.isEmpty || true {
            VStack(alignment: .leading, spacing: 8) {
                Divider()

                HStack(spacing: 8) {
                    Label("Attachments", systemImage: "paperclip")
                        .font(.headline)
                        .foregroundColor(.secondary)

                    Spacer()

                    Button(action: addAttachment) {
                        Image(systemName: "plus")
                            .font(.caption)
                            .padding(6)
                            .background(Color.secondary.opacity(0.2))
                            .clipShape(Circle())
                    }
                        .buttonStyle(.plain)
                        .help("Add attachment")
                }

                if !sortedAttachments.isEmpty {
                    ScrollView(.horizontal, showsIndicators: false) {
                        HStack(spacing: 8) {
                            ForEach(sortedAttachments) { attachment in
                                AttachmentThumbnailView(attachment: attachment)
                                    .contextMenu {
                                        Button("Open") { openAttachment(attachment) }
                                        Button("Quick Look") { quickLookAttachment(attachment) }
                                        if AttachmentManager.isImage(attachment.mediaType) {
                                            Button("Insert Inline") { insertInline(attachment) }
                                        }
                                    }
                            }
                            Divider()
                            Button("Delete", role: .destructive) { deleteAttachment(attachment) }
                        }
                    }
                }
            }
        }
    }
}
```

```

}
}
}
}
.padding(.horizontal, 16)
.padding(.bottom, 8)
.onDrop(of: [.fileURL], isTargeted: $dragOver) { providers in
    handleDrop(providers)
    return true
}
.overlay(
    dragOver ? RoundedRectangle(cornerRadius: 8)
    .stroke(Color.accentColor, lineWidth: 2)
    .padding(2) : nil
)
}
}

private func addAttachment() {
    let panel = NSOpenPanel()
    panel.allowsMultipleSelection = true
    panel.canChooseDirectories = false
    panel.allowedContentTypes = [.image, .movie, .audio, .pdf,
        .init("com.microsoft.word.doc") ?? .data,
        .init("org.openxmlformats.wordprocessingml.document") ?? .data,
        .init("com.microsoft.excel.xls") ?? .data,
        .init("org.openxmlformats.spreadsheetml.sheet") ?? .data,
        .init("com.microsoft.powerpoint.ppt") ?? .data,
        .init("org.openxmlformats.presentationml.presentation") ?? .data]

    guard panel.runModal() == .OK else { return }

    for url in panel.urls {
        addFile(at: url)
    }
}

private func addFile(at url: URL) {
    do {
        let attachment = try AttachmentManager.storeFile(at: url, for: prompt.id)
        attachment.sortOrder = prompt.attachments.count

        // Generate thumbnail for images
        if AttachmentManager.isImage(attachment.mediaType) {
            let resolvedURL = AttachmentManager.resolveURL(for: attachment) ?? url
            attachment.thumbnailData = AttachmentManager.generateThumbnail(for: resolvedURL)
        }

        modelContext.insert(attachment)
        prompt.attachments.append(attachment)
        prompt.updatedAt = Date()
        try modelContext.save()
    } catch {
        // Log error silently – could show alert in future
    }
}

private func handleDrop(_ providers: [NSItemProvider]) {

```

```

for provider in providers {
provider.loadItem(forTypeIdentifier: UTType.fileURL.identifier, options: nil) { data, _
in
guard let data = data as? Data,
let url = URL(dataRepresentation: data, relativeTo: nil) else { return }
DispatchQueue.main.async {
addFile(at: url)
}
}
}
}

private func deleteAttachment(_ attachment: Attachment) {
AttachmentManager.deleteFile(for: attachment)
prompt.attachments.removeAll { $0.id == attachment.id }
modelContext.delete(attachment)
prompt.updatedAt = Date()
try? modelContext.save()
}

private func openAttachment(_ attachment: Attachment) {
guard let url = AttachmentManager.resolveURL(for: attachment) else { return }
NSWorkspace.shared.open(url)
}

private func quickLookAttachment(_ attachment: Attachment) {
guard let url = AttachmentManager.resolveURL(for: attachment) else { return }
NSWorkspace.shared.activateFileViewerSelecting([url])
}

private func insertInline(_ attachment: Attachment) {
// Post notification – RichTextEditor listens and inserts the image
guard let url = AttachmentManager.resolveURL(for: attachment) else { return }
NotificationCenter.default.post(
name: .insertInlineImage,
object: nil,
userInfo: ["url": url]
)
}

private struct AttachmentThumbnailView: View {
let attachment: Attachment

var body: some View {
VStack(spacing: 4) {
if let thumbnailData = attachment.thumbnailData,
let nsImage = UIImage(data: thumbnailData) {
Image(nsImage: nsImage)
.resizable()
.aspectRatio(contentMode: .fill)
.frame(width: 80, height: 80)
.clipShape(RoundedRectangle(cornerRadius: 6))
} else {
ZStack {
RoundedRectangle(cornerRadius: 6)
.fill(Color.secondary.opacity(0.1))
Image(systemName: iconForMediaType(attachment.mediaType))
}
}
}
}

```



```

        .font(.title2)
        .foregroundColor(.secondary)
    }
    .frame(width: 80, height: 80)
}

Text(attachment.filename)
    .font(.caption2)
    .lineLimit(1)
    .truncationMode(.middle)
    .frame(width: 80)
}
}

private func iconForMediaType(_ type: String) -> String {
    if AttachmentManager.isImage(type) { return "photo" }
    if type.contains("movie") || type.contains("video") { return "film" }
    if type.contains("audio") { return "waveform" }
    if type.contains("pdf") { return "doc.richtext" }
    if type.contains("word") || type.contains("document") { return "doc.text" }
    if type.contains("excel") || type.contains("spreadsheet") { return "tablecells" }
    if type.contains("powerpoint") || type.contains("presentation") { return
        "rectangle.on.rectangle" }
    return "doc"
}
}
}

```

Step 2: Add the notification name

In Pault/PaultApp.swift:

```

extension Notification.Name {
    static let createNewPrompt = Notification.Name("com.pault.createNewPrompt")
    static let promptCreated = Notification.Name("com.pault.promptCreated")
    static let insertInlineImage = Notification.Name("com.pault.insertInlineImage")
}

```

Step 3: Embed AttachmentsStripView in PromptDetailView

In Pault/PromptDetailView.swift, add after the TemplateVariablesView:

```

AttachmentsStripView(prompt: prompt)

```

Step 4: Build and verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 5: Commit

```

git add Pault/AttachmentsStripView.swift Pault/PaultApp.swift
Pault/PromptDetailView.swift
git commit -m "feat: add attachments strip with drag-drop, thumbnails, and context menu"

```

Task 13: Upgrade clipboard to rich copy (RTFD + plain text)

Files:

- Modify: Pault/PromptService.swift:58-64

Step 1: Update copyToClipboard

In `Pault/PromptService.swift`, replace the `copyToClipboard` method:

```
func copyToClipboard(_ prompt: Prompt) {
    let resolved = TemplateEngine.resolve(content: prompt.content, variables:
prompt.templateVariables)
    let pasteboard = NSPasteboard.general
    pasteboard.clearContents()

    // Always provide plain text
    pasteboard.setString(resolved, forType: .string)

    // If rich content exists, also provide RTFD
    if let rtfdData = prompt.attributedContent,
        let attrString = try? NSAttributedString(data: rtfdData, options: [.documentType:
NSAttributedString.DocumentType.rtf], documentAttributes: nil) {
        // Resolve template variables in the attributed string
        let mutable = NSMutableAttributedString(attributedString: attrString)
        let fullString = mutable.string
        let resolvedAttr = TemplateEngine.resolve(content: fullString, variables:
prompt.templateVariables)
        // Only replace text portions, preserving attachments at their positions
        if let rtfdOutput = try? mutable.data(from: NSRange(location: 0, length:
mutable.length), documentAttributes: [.documentType:
NSAttributedString.DocumentType.rtf]) {
            pasteboard.setData(rtfdOutput, forType: .rtfd)
        }
    }

    prompt.markAsUsed()
    save("copyToClipboard")
}
```

Step 2: Build and run tests

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' -only-testing PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 3: Commit

```
git add Pault/PromptService.swift
git commit -m "feat: upgrade clipboard to include RTFD with inline images"
```

Task 14: Clean up PromptService.deletePrompt to remove files

Files:

- Modify: `Pault/PromptService.swift:37-40`

Step 1: Update deletePrompt to clean up disk files

In `Pault/PromptService.swift`, update `deletePrompt`:

```
func deletePrompt(_ prompt: Prompt) {
    AttachmentManager.deleteFiles(for: prompt.id)
    modelContext.delete(prompt)
    save("deletePrompt")
}
```

Step 2: Build and verify

Run: `xcodebuild build -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' 2>&1 | tail -5`

Expected: BUILD SUCCEEDED

Step 3: Commit

```
git add Pault/PromptService.swift
git commit -m "fix: clean up attachment files on prompt deletion"
```

Task 15: Final integration test and cleanup**Step 1: Run all tests**

Run: `xcodebuild test -project Pault.xcodeproj -scheme Pault -destination 'platform=macOS' -only-testing PaultTests 2>&1 | tail -20`

Expected: All tests pass

Step 2: Manual testing checklist

- [] Cmd+N opens new creation window
- [] Fill title, content, select tags, create prompt — prompt appears in sidebar
- [] Cancel dismisses without creating
- [] Variable fields auto-expand with text
- [] Variable preview shows resolved output
- [] Click "+" in attachments strip → file picker opens
- [] Drag files from Finder → attachments strip accepts them
- [] Image thumbnails appear in strip
- [] Non-image files show icon placeholder
- [] Right-click attachment → Open, Quick Look, Delete work
- [] Copy prompt → plain text in clipboard
- [] Copy prompt with rich content → RTFD in clipboard (paste into TextEdit to verify)
- [] Delete prompt → attachment files removed from disk
- [] Template variables still parse and fill correctly
- [] Search still works (uses plain text content)
- [] Menu bar and hotkey launcher still copy plain text only

Step 3: Final commit

```
git commit --allow-empty -m "checkpoint: all three features complete – creation window,
expanding fields, rich media"
```

Files Summary

| File | Action | Feature |

|-----|-----|-----|

| Pault/ExpandingTextEditor.swift | **CREATE** | F2 — auto-expanding fields |

| Pault/TemplateVariablesView.swift | **MODIFY** | F2 — use ExpandingTextEditor |

Pault/NewPromptView.swift	**CREATE**	F1 — creation window
Pault/TagPickerPopover.swift	**CREATE**	F1 — shared tag picker
Pault/PaultApp.swift	**MODIFY**	F1 — add Window scene, notifications
Pault/ContentView.swift	**MODIFY**	F1 — open window instead of inline create
Pault/InspectorView.swift	**MODIFY**	F1 — use shared TagPickerPopover
Pault/Attachment.swift	**CREATE**	F3 — attachment model
Pault/AttachmentManager.swift	**CREATE**	F3 — file storage
Pault/RichTextEditor.swift	**CREATE**	F3 — rich text editing
Pault/AttachmentsStripView.swift	**CREATE**	F3 — attachment strip UI
Pault/Prompt.swift	**MODIFY**	F3 — add attachments, attributedContent
Pault/PromptDetailView.swift	**MODIFY**	F3 — embed rich editor and strip
Pault/PromptService.swift	**MODIFY**	F3 — rich clipboard, file cleanup
PaultTests/AttachmentTests.swift	**CREATE**	F3 — model tests
PaultTests/AttachmentManagerTests.swift	**CREATE**	F3 — storage tests

Verification

1. **Build:** xcodebuild build succeeds after each task
2. **Unit tests:** All PaultTests pass (template engine, model, attachment tests)
3. **Manual:** Creation window, expanding fields, attachments, clipboard all work
4. **Regression:** Search, menu bar, hotkey launcher, inspector unchanged